CyberSOC Platform

Comprehensive Test Suite & Purple Team Validation Framework

Phase 4: Go-Live Roadmap Execution

Document Type	Technical Testing Framework
Version	1.0.0 - Production Ready
Classification	Internal Technical / Security Sensitive
Date	2026-08-26
Scope	Full Platform Quality Assurance Program
Target Audience	QA Engineers, SRE, Security Teams, DevOps

Table of Contents

1. Testing Strategy Overview

The CyberSOC Platform testing strategy implements a comprehensive quality assurance program spanning multiple testing disciplines, execution environments, and maturity levels to ensure production readiness across functional correctness, security posture, performance characteristics, and operational resilience. The strategy follows the testing pyramid model with extensive unit test coverage forming the foundation, progressive integration testing validating component interactions, and targeted end-to-end scenarios confirming complete user workflows. Security testing receives dedicated emphasis given the platform's role as a security operations tool where trust depends on demonstrable security competence.

1.1 Testing Philosophy and Principles

The testing philosophy centers on shift-left principles integrating quality activities early in the development lifecycle when defect remediation costs remain lowest. Automated testing forms the backbone of the quality program with manual testing reserved for exploratory scenarios requiring human judgment, usability evaluation, and complex security assessments where automated tools provide incomplete coverage. Test-driven development (TDD) practices encourage writing failing tests before implementation code, ensuring comprehensive requirement coverage and serving as executable documentation that remains synchronized with implementation changes.

1.2 Testing Pyramid and Coverage Targets

Test Layer	Coverage Target	Automation %	Execution Frequency	Owner
Unit Tests	> 85% line coverage	100%	Every commit (CI)	Developers
Integration Tests	> 75% service paths	95%	Every PR + daily	DevOps/SRE
E2E Tests	Critical paths only	80%	Pre-merge + nightly	QA Team
Performance Tests	Key transactions	90%	Weekly + pre-release	Performance Eng
Security Tests	OWASP Top 10 + Custom	70%	Weekly + on release	AppSec/Red Team
Chaos Tests	Failure modes	60%	Monthly + on change	SRE/Platform
Manual/Exploratory	Usability + Edge cases	0%	Sprint cycles	QA Analysts

Table 1.1: Testing Pyramid Coverage Requirements

1.3 Test Environment Strategy

Multiple test environments support different testing phases with appropriate data volumes, configuration states, and isolation guarantees. Development environments provide rapid feedback for individual developers running localized tests against mocked dependencies. Shared integration environments validate service interactions using containerized dependency stubs or shared instances with deterministic test data. Staging environments mirror production

configuration including scaled-down infrastructure, real identity providers in test mode, and anonymized production data subsets enabling realistic performance characterization. Production-like performance environments execute load tests without affecting user-facing systems while maintaining representative network topology and data distributions.

Environment	Purpose	Data Source	Access	Refresh Cadence
Local Dev	Unit + Component dev	Mocked/synthetic	Developer only	On-demand
CI/CD Pipeline	Automated gate checks	Fixtures/seeds	Automated only	Per build
Dev Integration	Cross-service testing	Test datasets	Dev team	Daily reset
Staging	Pre-release validation	Anonymized prod subset	QA + Dev	Weekly refresh
Perf Testing	Load/stress execution	Scaled synthetic	Perf team	Per test cycle
Security Lab	Pen testing, vuln scan	Sensitive test data	Red team only	Isolated

Table 1.2: Test Environment Matrix

2. Unit Testing Framework

Unit testing validates individual functions, methods, and classes in isolation from external dependencies through mocking and stubbing techniques. The CyberSOC platform utilizes Jest for frontend React/Vue components, pytest for Python backend services including threat detection engines and SIEM correlation logic, and Go testing packages for infrastructure tools and Kubernetes operators. Unit test suites serve dual purposes as regression safety nets preventing unintended behavior changes and as living documentation expressing expected component behavior through concrete examples.

2.1 Frontend Unit Testing (Jest + React Testing Library)

Frontend unit tests focus on component rendering correctness, user interaction handling, state management integration, and accessibility compliance. React Testing Library encourages testing user behavior rather than implementation details, ensuring tests survive refactoring while catching meaningful regressions. Accessibility assertions verify ARIA attributes, keyboard navigation support, and screen reader compatibility essential for enterprise software serving diverse user populations including security analysts who may rely on assistive technologies during extended monitoring sessions.

```
// Example: Threat Intelligence Card Component Test
import { render, screen, fireEvent } from '@testing-library/react';
import { ThreatIntelCard } from './ThreatIntelCard';

describe('ThreatIntelCard Component', () => {
  const mockThreatData = {
    id: 'THREAT-001',
    title: 'APT29 Phishing Campaign',
    severity: 'critical',
    iocCount: 47,
```

```
    lastUpdated: '2024-11-15T10:30:00Z',
    tags: ['phishing', 'apt29', 'credential-theft']
  };

  it('renders threat title and severity badge correctly', () => {
    render(<ThreatIntelCard threat={mockThreatData} />);
    expect(screen.getByText('APT29 Phishing Campaign')).toBeInTheDocument();
    expect(screen.getByTestId('severity-badge')).toHaveClass('severity-critical');
  });

  it('displays IOC count with proper formatting', () => {
    render(<ThreatIntelCard threat={mockThreatData} />);
    expect(screen.getByText(/47 Indicators/i)).toBeInTheDocument();
  });

  it('calls onClick handler when card is clicked', () => {
    const mockOnClick = jest.fn();
    render(<ThreatIntelCard threat={mockThreatData} onClick={mockOnClick} />);
    fireEvent.click(screen.getByRole('button'));
    expect(mockOnClick).toHaveBeenCalledWith('THREAT-001');
  });

  it('applies accessible labels for screen readers', () => {
    render(<ThreatIntelCard threat={mockThreatData} />);
    expect(screen.getByLabelText(/threat intelligence card/i)).toBeInTheDocument();
  });
});
```

Listing 2.1: Frontend Component Unit Test Example

2.2 Backend Unit Testing (pytest)

Backend unit tests validate business logic including threat scoring algorithms, SIEM event correlation rules, access control policy evaluation, and data transformation pipelines. Pytest fixtures provide reusable test context objects including mock database sessions, authenticated user representations, and sample event payloads covering diverse log formats. Parameterized tests efficiently validate algorithm behavior across input boundary conditions, edge cases, and representative samples drawn from production event distributions ensuring statistical coverage of real-world scenarios.

```
# Example: Threat Scoring Algorithm Unit Test
import pytest
from cybersoc.engine.threat_scorer import ThreatScorer, ThreatContext

@pytest.fixture
def scorer():
    return ThreatScorer(config_path='tests/fixtures/scorer_config.yaml')

class TestThreatScorer:
    def test_critical_score_for_known_apt_ioc(self, scorer):
        context = ThreatContext(
            ioc_source='known_apt_feed',
            target_matches=True,
            historical_attacks=5,
            time_decay_factor=0.9
        )
        score = scorer.calculate(context)
        assert score.severity == 'CRITICAL'
        assert score.numeric >= 9.0

    @pytest.mark.parametrize("ioc_count,expected_range", [
```

```
    (1, (1.0, 3.0)),
    (10, (3.0, 5.0)),
    (50, (5.0, 7.0)),
    (200, (7.0, 9.0)),
])
def test_score_scales_with_ioc_count(self, scorer, ioc_count, expected_range):
    context = ThreatContext(ioc_count=ioc_count)
    score = scorer.calculate(context)
    assert expected_range[0] <= score.numeric <= expected_range[1]

def test_time_decay_reduces_old_threat_scores(self, scorer):
    recent_context = ThreatContext(hours_since_seen=1)
    old_context = ThreatContext(hours_since_seen=720)  # 30 days

    recent_score = scorer.calculate(recent_context)
    old_score = scorer.calculate(old_context)

    assert recent_score.numeric > old_score.numeric

def test_handles_empty_context_gracefully(self, scorer):
    empty_context = ThreatContext()
    score = scorer.calculate(empty_context)
    assert score.severity == 'LOW'
    assert score.numeric >= 0
```

Listing 2.2: Backend Business Logic Unit Test Example

3. Integration Testing Procedures

Integration testing validates correct interaction between CyberSOC platform components including API contracts between microservices, database query patterns producing expected results, message queue consumption and production flows, cache invalidation cascades, and external service integrations with identity providers, threat intelligence feeds, and notification systems. Unlike unit tests that isolate components through mocking, integration tests exercise real component connections against containerized or shared test infrastructure, revealing compatibility issues, protocol misunderstandings, and timing-dependent bugs invisible in isolated testing.

3.1 API Contract Testing

API contract testing ensures service implementations conform to agreed-upon interfaces defined in OpenAPI/Swagger specifications, preventing breaking changes from propagating undetected through dependent services. Consumer-driven contract testing captures each consumer's expectations in contract files verified against provider implementations during CI/CD pipelines. Pact (for HTTP APIs) and AsyncAPI (for message-based integrations) provide mature tooling supporting contract generation, versioning, and verification with clear failure messages identifying specific specification violations requiring remediation.

Service Pair	Contract Tool	Verification Point	Frequency	Owner
Gateway -> Auth	Pact	Auth token format, error codes	PR + daily	Platform Team
Gateway -> Threat Engine	OpenAPI	Request/response schemas	PR + daily	API Team

Service Pair	Contract Tool	Verification Point	Frequency	Owner
SIEM -> PostgreSQL	SQL lint	Query syntax, index usage	PR only	DBA Team
UI -> Gateway	MSW mocks	HTTP interface contract	PR only	Frontend Team
Engine -> Redis	Redis-spec	Cache key patterns, TTL	PR only	Backend Team
All -> Elasticsearch	ES schema	Index mappings, queries	Pre-release	Data Team

Table 3.1: API Contract Testing Matrix

3.2 Database Integration Testing

Database integration tests validate ORM/ODM layer functionality against real database instances ensuring query generation produces efficient SQL, migrations apply cleanly without data loss, transaction boundaries maintain consistency under concurrent access, and connection pooling handles load patterns without exhaustion. Testcontainers library provides ephemeral database instances spun up within test execution, enabling true integration testing without shared test database contention or stale data interference between parallel test runs. Each test method executes within transactional boundaries rolled back after assertion completion, maintaining test isolation while exercising full persistence stack including trigger execution and constraint enforcement.

```
# Example: Incident Repository Integration Test
import pytest
from testcontainers.postgresql import PostgresqlContainer
from sqlalchemy import create_engine
from cybersoc.db.repositories import IncidentRepository
from cybersoc.models.incident import Incident, IncidentStatus

@pytest.fixture(scope="class")
def pg_container():
    with PostgresqlContainer("postgres:16-alpine") as pg:
        yield pg

@pytest.mark.integration
class TestIncidentRepository:
    @pytest.fixture(autouse=True)
    def setup(self, pg_container):
        engine = create_engine(pg.get_connection_url())
        self.repo = IncidentRepository(engine)
        # Run migrations
        Base.metadata.create_all(engine)
        yield
        Base.metadata.drop_all(engine)

    def test_create_and_retrieve_incident(self):
        incident = Incident(
            title="Phishing campaign detected",
            severity="high",
            status=IncidentStatus.OPEN,
            description="User reported suspicious email"
        )
        created = self.repo.create(incident)

        retrieved = self.repo.get_by_id(created.id)
        assert retrieved.title == "Phishing campaign detected"
```

```
        assert retrieved.status == IncidentStatus.OPEN
        assert created.created_at is not None

    def test_query_by_status_with_pagination(self):
        # Create test data
        for i in range(25):
            status = IncidentStatus.OPEN if i < 15 else IncidentStatus.CLOSED
            self.repo.create(Incident(title=f"Incident-{i}", status=status))

        open_incidents = self.repo.query(status=IncidentStatus.OPEN, page=1, per_page=10)
        assert len(open_incidents.items) == 10
        assert open_incidents.total == 15
        assert open_incidents.pages == 2
```

Listing 3.1: Database Integration Test with Testcontainers

4. End-to-End Testing Scenarios

End-to-end (E2E) tests validate complete user journeys traversing multiple system components from initial user action through backend processing to final result presentation, simulating real usage patterns that expose integration issues invisible at lower testing levels. E2E test suites prioritize critical business paths including authentication flows, core workflow execution (threat investigation, incident response case lifecycle), reporting generation, and administrative operations. Playwright provides cross-browser E2E execution capabilities with automatic waiting, retry logic, and screenshot capture on failure enabling efficient debugging of flaky tests often plaguing UI automation efforts.

4.1 Critical User Journey Test Cases

Scenario ID	User Journey	Components Tested	Priority	Execution
E2E-001	Login -> Dashboard -> Logout	Auth, Gateway, UI, Session	P0	Every merge
E2E-002	Create Incident -> Assign -> Resolve	Case Mgmt, Notifications	P0	Every merge
E2E-003	Ingest Event -> Correlate -> Alert	SIEM, Rules Engine, Alerts	P0	Nightly
E2E-004	Run Investigation Report -> Export PDF	Reporting, File Gen, Storage	P1	Pre-release
E2E-005	Admin: Add User -> Configure RBAC	Admin API, IAM, IdP Sync	P1	Pre-release
E2E-006	Configure Threat Feed -> Validate IOCs	TI Module, Parser, DB	P1	Weekly
E2E-007	Search Historical Events -> Drill Down	ES Query, UI Pagination	P2	Weekly
E2E-008	Mobile Responsive: View Alerts	Responsive UI, API	P2	Bi-weekly

Table 4.1: Critical E2E Test Scenario Inventory

4.2 Playwright E2E Test Implementation

```
// Example: Incident Management E2E Test
import { test, expect } from '@playwright/test';
```

```
test.describe('Incident Management Workflow', () => {
  test.beforeEach(async ({ page }) => {
    // Authenticate via API token
    await page.goto('/login');
    await page.fill('[data-testid="email"]', 'analyst@cybersoc.test');
    await page.fill('[data-testid="password"]', 'test-password');
    await page.click('[data-testid="login-button"]');
    await page.waitForURL('/dashboard');
  });

  test('should create new incident and verify in list', async ({ page }) => {
    // Navigate to incidents
    await page.click('[data-testid="nav-incidents"]');
    await expect(page).toHaveURL(/\/incidents/);

    // Click create button
    await page.click('[data-testid="create-incident"]');
    await expect(page.locator('.modal')).toBeVisible();

    // Fill incident form
    await page.fill('[name="title"]', 'Test E2E Incident');
    await page.selectOption('[name="severity"]', 'high');
    await page.fill('[name="description"]', 'Automated E2E test incident');

    // Submit form
    await page.click('[data-testid="submit-incident"]');

    // Verify success
    await expect(page.locator('.toast-success')).toContainText('created');

    // Verify incident appears in list
    await expect(page.locator('text=Test E2E Incident')).toBeVisible({ timeout: 5000 });
  });

  test('should assign incident and update status', async ({ page }) => {
    // Find existing incident
    await page.goto('/incidents');
    const incidentRow = page.locator(`tr[data-severity="high"]`).first();
    await incidentRow.click();

    // Assign to analyst
    await page.click('[data-testid="assign-button"]');
    await page.selectOption('[name="assignee"]', 'analyst-user-id');
    await page.click('[data-testid="confirm-assign"]');

    // Update status to In Progress
    await page.selectOption('[name="status"]', 'in_progress');
    await page.click('[data-testid="update-status"]');

    // Verify updates
    await expect(page.locator('[data-testid="current-status"]')).toHaveText('In Progress');
    await expect(page.locator('[data-testid="assignee-name"]')).toHaveText('Analyst User');
  });
});
```

Listing 4.1: Playwright E2E Test Implementation

5. Performance and Load Testing

Performance testing validates system responsiveness, throughput capacity, and resource utilization under various load conditions from normal operation through peak demand spikes to stress conditions exceeding design limits. The performance testing program establishes baseline metrics for comparison during regression testing, identifies bottlenecks limiting scalability, validates auto-scaling configurations, and provides confidence that production infrastructure sizing adequately supports projected customer workloads. K6 provides modern load testing capabilities with JavaScript test scripts, cloud execution for distributed load generation, and result visualization integrated with Grafana dashboards for trend analysis.

5.1 Performance Test Types and Objectives

Test Type	Objective	Load Profile	Duration	Success Criteria
Baseline	Establish reference metrics	10% of expected peak	1 hour	Record response times
Load Test	Validate SLA compliance	Expected peak load	2-4 hours	P99 < 2000ms, 0% errors
Stress Test	Find breaking point	Incremental to failure	Gradual ramp	Identify max capacity
Soak Test	Detect memory leaks	Sustained 80% load	24-72 hours	No degradation over time
Spike Test	Handle sudden bursts	5-10x normal, short	15-30 min bursts	Auto-scale recovers
Scale Test	Validate horizontal scaling	Trigger HPA thresholds	Until stable	New pods serve traffic

Table 5.1: Performance Test Type Definitions

5.2 Key Performance Indicators and Thresholds

Metric	Component	Baseline Target	Warning Threshold	Critical Threshold
API Response P50	Gateway	< 100ms	> 150ms	> 300ms
API Response P99	Gateway	< 500ms	> 1000ms	> 2000ms
Event Ingestion Rate	SIEM Ingest	10,000 eps	< 8000 eps sustained	< 5000 eps
Correlation Latency	SIEM Engine	< 5 seconds	> 10 seconds	> 30 seconds
Search Response Time	Elasticsearch	< 2 seconds	> 5 seconds	> 10 seconds
Auth Token Issuance	Auth Service	< 200ms	> 500ms	> 1000ms
Dashboard Load	Frontend	< 3 seconds	> 5 seconds	> 8 seconds
Report Generation	Reporting Service	< 30 seconds	> 60 seconds	> 120 seconds
Concurrent Users	System	500 active	CPU > 80%	Memory > 90%
Error Rate	All Components	< 0.1%	> 0.5%	> 2%

Table 5.2: Performance KPI Thresholds by Component

5.3 K6 Load Test Script Example

```
// k6 Load Test Script: CyberSOC API Gateway
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

// Custom metrics
const errorRate = new Rate('errors');
const apiResponseTime = new Trend('api_response_time');

export const options = {
  stages: [
    { duration: '2m', target: 100 },   // Ramp-up
    { duration: '10m', target: 500 },   // Sustained load (normal peak)
    { duration: '5m', target: 1000 },   // Stress phase
    { duration: '2m', target: 0 },      // Recovery
  ],
  thresholds: {
    http_req_duration: ['p(99)<2000'],  // P99 < 2 seconds
    errors: ['rate<0.01'],               // < 1% error rate
  },
};

const BASE_URL = __ENV.BASE_URL || 'https://api.cybersoc.test';

export default function () {
  // Authentication flow
  const loginPayload = JSON.stringify({
    email: `loaduser_${__VU}-test@cybersoc.io`,
    password: 'load-test-password',
  });

  const loginRes = http.post(`${BASE_URL}/auth/login`, loginPayload, {
    headers: { 'Content-Type': 'application/json' },
  });

  check(loginRes, {
    'login successful': (r) => r.status === 200,
    'token received': (r) => r.json('accessToken') !== undefined,
  });

  const authToken = loginRes.json('accessToken');

  // Dashboard fetch (authenticated request)
  const dashboardRes = http.get(`${BASE_URL}/dashboard/metrics`, {
    headers: {
      'Authorization': `Bearer ${authToken}`,
      'Accept': 'application/json',
    },
  });

  errorRate.add(dashboardRes.status !== 200);
  apiResponseTime.add(dashboardRes.timings.duration);

  check(dashboardRes, {
    'dashboard loaded': (r) => r.status === 200,
    'response time acceptable': (r) => r.timings.duration < 2000,
    'metrics present': (r) => r.json('threatCount') !== undefined,
  });

  // Simulate think time between requests
  sleep(Math.random() * 3 + 1);  // 1-4 seconds
}

export function handleSummary(data) {
```

```
  console.log('\n=== Load Test Summary ===');
  console.log(`Total Requests: ${data.metrics.http_reqs.values.count}`);
  console.log(`Error Rate: ${(data.metrics.errors.rate * 100).toFixed(2)}%`);
  console.log(`P95 Response: ${data.metrics.http_req_duration.values['p(95)']}ms`);
  console.log(`P99 Response: ${data.metrics.http_req_duration.values['p(99)']}ms`);
}
```

Listing 5.1: K6 Load Test Script for API Gateway

6. Security Penetration Testing

Security penetration testing employs adversarial techniques to identify exploitable vulnerabilities before malicious actors discover them in production environments. The CyberSOC platform undergoes regular penetration testing combining automated vulnerability scanning with manual exploitation attempts by qualified security professionals. Testing scope encompasses the application layer (web API, web application, mobile interfaces if applicable), infrastructure layer (container images, Kubernetes configuration, cloud resources), and human factors (social engineering targeting personnel with platform access). Findings are classified by severity following CVSS v3.1 scoring with remediation timelines aligned to organizational vulnerability management policies.

6.1 Penetration Testing Methodology

The penetration testing methodology follows industry-standard approaches adapted for cloud-native security operations platforms. Reconnaissance phase maps attack surface including discovered endpoints, technology fingerprinting, and information leakage assessment. Vulnerability analysis combines automated scanning (OWASP ZAP, Burp Suite, Nessus) with manual review focusing on business logic flaws escaping automated detection. Exploitation phase demonstrates impact of confirmed vulnerabilities, progressing from unauthenticated through authenticated access to privilege escalation paths. Post-exploitation assesses lateral movement potential, data exfiltration feasibility, and persistence mechanism establishment providing realistic risk characterization for remediation prioritization decisions.

Phase	Activities	Tools	Deliverables	Duration
Reconnaissance	OSINT, endpoint discovery, footprinting	Recon-ng, Nmap, Amass	Attack surface map	2-3 days
Vulnerability Discovery	Auto-scan + manual analysis	ZAP, Burp, Semgrep	Finding list	5-7 days
Exploitation	Proof-of-concept exploits	Metasploit, Custom	POC demos	3-5 days
Post-Exploitation	Lateral move, data access	Cobalt Strike, Empire	Impact assessment	2-3 days
Reporting	Executive + technical reports	Dradis, Custom templates	Final report	3-5 days
Remediation Support	Retesting, guidance	Same as above	Verified closure	2-3 days

Table 6.1: Penetration Testing Phase Breakdown

6.2 OWASP Top 10 Coverage Matrix

OWASP Category	Test Cases	Automated	Last Test Date	Findings
A01: Broken Access Control	12 test cases	Partial	2024-Q3	1 Low
A02: Cryptographic Failures	8 test cases	Yes	2024-Q3	0
A03: Injection	15 test cases	Yes	2024-Q3	0
A04: Insecure Design	6 test cases	No	2024-Q2	2 Medium
A05: Security Misconfiguration	20 test cases	Yes	2024-Q3	3 Low
A06: Vulnerable Components	10 test cases	Yes	2024-Q3	1 Medium
A07: Auth Failures	14 test cases	Partial	2024-Q3	0
A08: Software/Data Integrity	8 test cases	Partial	2024-Q2	1 Low
A09: Logging/Monitoring Failures	7 test cases	No	2024-Q2	2 Low
A10: SSRF	9 test cases	Partial	2024-Q3	0

Table 6.2: OWASP Top 10 Testing Coverage Status

7. Purple Team Validation Exercises

Purple team exercises represent collaborative security assessments where offensive security professionals (red team) and defensive security teams (blue team) work together to validate detection and response capabilities against simulated adversary techniques. Unlike traditional penetration testing focused on breach demonstration, purple team exercises emphasize measurable improvement of defensive controls through controlled scenario execution with real-time visibility into how attacks manifest in logging, alerting, and analyst detection workflows. Each exercise maps to MITRE ATT&CK; techniques enabling systematic coverage of adversary tactics relevant to security operations platform threats.

7.1 ATT&CK; Technique Coverage Plan

The MITRE ATT&CK; framework provides a structured knowledge base of adversary tactics, techniques, and procedures (TTPs) observed in real-world attacks. Purple team exercises systematically validate detection coverage for techniques most likely targeting security operations platforms including initial access vectors, credential theft mechanisms, lateral movement paths, defense evasion tactics, and data exfiltration methods. Each technique receives a detection coverage rating (Detected, Partially Detected, Not Detected) based on exercise outcomes, informing investment priorities for security control enhancement.

Tactic	Technique ID	Technique Name	Detection Status	Last Validated
Initial Access	T1078	Valid Accounts	DETECTED	2024-Q3
Initial Access	T1190	Exploit Public-Facing App	DETECTED	2024-Q3

Tactic	Technique ID	Technique Name	Detection Status	Last Validated
Credential Access	T1110.001	Brute Force - Password Guessing	DETECTED	2024-Q3
Credential Access	T1528	Steal Application Token	PARTIAL	2024-Q2
Discovery	T1082	System Information Discovery	DETECTED	2024-Q3
Discovery	T1046	Network Service Discovery	PARTIAL	2024-Q2
Lateral Movement	T1021.004	Remote Services - SSH	DETECTED	2024-Q3
Lateral Movement	T1570	Lateral Tool Transfer	NOT DETECTED	2024-Q2
Collection	T1005	Data from Local System	DETECTED	2024-Q3
Exfiltration	T1048	Exfiltration Over Alternative Protocol	PARTIAL	2024-Q2
Impact	T1486	Data Encrypted for Impact	DETECTED	2024-Q3

Table 7.1: MITRE ATT&CK; Detection Coverage Matrix

7.2 Purple Team Exercise Scenarios

Exercise ID	Scenario Name	ATT&CK Techniques	Objective	Duration
PT-001	Credential Stuffing Attack	T1110, T1110.004	Validate brute force detection	4 hours
PT-002	Insider Threat - Data Exfil	T1005, T1048, T1567	DLP effectiveness test	6 hours
PT-003	Supply Chain Compromise	T1195, T1078	Third-party risk validation	8 hours
PT-004	Ransomware Simulation	T1486, T1490	IR playbook validation	4 hours
PT-005	Privilege Escalation Path	T1068, T1548	PAM control verification	4 hours
PT-006	API Abuse & Rate Limit Bypass	T1119, T1499	WAF/gateway testing	3 hours
PT-007	Session Hijacking	T1563, T1550	Session security validation	3 hours
PT-008	Persistence Mechanism	T1543, T1546	Detection of persistence	4 hours

Table 7.2: Purple Team Exercise Scenario Catalog

7.3 Exercise Execution Template

Each purple team exercise follows a standardized execution template ensuring consistent documentation, reproducible results, and actionable findings. Pre-exercise preparation includes scenario briefing to all participants, environment backup for clean recovery, and alert suppression coordination to prevent automated responses interfering with controlled execution. During exercise execution, red team actions are logged with timestamps, blue team detection observations are recorded with alert IDs and investigation notes, and command-and-control communication maintains scenario scope boundaries preventing unintended production impact. Post-exercise analysis compares red team action timeline

against blue team detection timeline calculating mean time to detect (MTTD) and mean time to respond (MTTR) metrics feeding continuous improvement initiatives.

8. Chaos Engineering Tests

Chaos engineering proactively validates system resilience by injecting controlled failures into production-like environments, measuring how gracefully the system degrades and recovers. For the CyberSOC platform, chaos experiments validate high availability architecture assumptions, auto-scaling responsiveness, failover mechanism correctness, and operational runbook effectiveness. Experiments follow the steady-state hypothesis approach defining normal operating conditions, injecting specific failure modes, and verifying the system returns to acceptable steady-state within defined recovery time objectives. All experiments require explicit approval, include blast radius limitations preventing customer impact, and feature immediate abort capabilities if unexpected degradation occurs.

8.1 Chaos Experiment Categories

Category	Experiment Examples	Tools	Risk Level	Frequency
Infrastructure	Node termination, AZ failure	Chaos Monkey, AWS FIS	Medium	Monthly
Network	Latency injection, packet loss	Toxiproxy, Istio Fault Injection	Low	Bi-weekly
Resource	CPU starvation, memory pressure	stress-ng, Linux tc	Low	Monthly
Application	Pod kill, process crash	Chaos Mesh, Litmus	Medium	Monthly
State	Disk fill, corruption simulation	Custom scripts	High	Quarterly
Dependency	DB connection drop, DNS failure	Gremlin, Custom	Medium	Quarterly
Time	Clock skew, certificate expiry	Libfaketime, Custom	Low	Quarterly

Table 8.1: Chaos Experiment Categories

8.2 Steady-State Hypotheses and Validation

Each chaos experiment defines a steady-state hypothesis representing the expected normal behavior that should persist despite injected failures. Hypotheses must be measurable through existing observability metrics (Prometheus queries, log patterns, synthetic checks) enabling automated validation during experiment execution. Failed hypotheses indicate resilience gaps requiring architectural improvements, enhanced monitoring, or updated runbooks. Hypothesis formulation involves collaboration between SRE teams understanding system behavior, development teams knowing implementation details, and product teams defining acceptable degradation boundaries from customer experience perspective.

Experiment	Steady-State Hypothesis	Validation Metric	Min Success Rate
Random Pod Termination	API availability remains > 99%	Success rate from LB health checks	95%
AZ Failure Simulation	RTO < 5 minutes, zero data loss	Recovery time, RPO measurement	90%
Network Latency (+200ms)	P99 latency increase < 300%	Apdex score degradation	85%
Memory Pressure (85%)	No OOM kills, graceful degradation	OOM count, error rate	100%
DB Primary Failure	Failover < 30s, zero lost writes	Failover time, WAL position	95%
DNS Resolution Failure	Circuit breaker opens, cached responses	Error rate, fallback success	90%
Certificate Expiry Imminent	Auto-renewal completes before expiry	Cert validity, renewal logs	100%

Table 8.2: Experiment Hypotheses and Validation Criteria

9. Test Automation and CI/CD Integration

Test automation embedded within CI/CD pipelines provides rapid feedback on code quality changes, preventing defect introduction and maintaining cumulative quality standards as the codebase evolves. The pipeline implements quality gates at multiple stages with escalating test breadth and execution time, balancing fast feedback for developer productivity with thorough validation protecting production deployments. Pipeline stages progress from quick unit test execution through integration testing against containerized services to comprehensive end-to-end validation against staging environments, with each stage gating promotion to subsequent stages upon passing defined threshold criteria.

9.1 Pipeline Stage Definitions

Stage	Trigger	Tests Executed	Duration	Gate Criteria
Pre-commit	Git hook	Lint, format, unit subset	< 2 minutes	Zero failures
CI Build	Push to main/PR	Full unit suite, SAST	< 10 minutes	> 85% coverage, 0 crit/high
Integration	CI passes	Integration tests, contract	< 20 minutes	> 90% pass rate
Staging Deploy	Merge to release	E2E smoke, perf baseline	< 30 minutes	P0 scenarios pass
Pre-production	Release candidate	Full E2E, security scan	< 60 minutes	All P0/P1 pass
Production Gate	Manual approval	Final validation checklist	< 15 minutes	Sign-off obtained

Table 9.1: CI/CD Pipeline Stage Configuration

9.2 Quality Gate Metrics and Thresholds

Quality gates define objective criteria determining pipeline progression, removing subjective judgment from deployment decisions while establishing documented quality floors for

production-bound code. Metrics combine absolute thresholds (minimum coverage percentages, maximum allowed vulnerabilities) with trend analysis (regression detection comparing current values against historical baselines) identifying gradual quality erosion before it crosses critical thresholds. Gate violations produce clear diagnostic information pinpointing specific test failures, coverage gaps, or metric deviations requiring remediation before pipeline continuation.

Metric	Unit Test Gate	Integration Gate	E2E Gate	Production Gate
Line Coverage	> 85%	> 80%	N/A	N/A
Branch Coverage	> 75%	> 70%	N/A	N/A
New Code Coverage	> 90%	> 85%	N/A	N/A
Test Pass Rate	100%	> 98%	> 95%	100% (P0/P1)
Critical Vulns	0	0	0	0
High Vulns	0	< 2 known	< 1 new	0
Flaky Test Rate	< 2%	< 3%	< 5%	N/A
Test Execution Time	< 10 min	< 20 min	< 45 min	< 15 min
Performance Regression	N/A	< 10% degrade	< 5% degrade	< 2% degrade

Table 9.2: Quality Gate Thresholds by Pipeline Stage

10. Test Data Management

Test data management addresses the challenge of providing realistic, varied, and privacy-compliant data for testing purposes without risking exposure of sensitive production information or introducing test data contamination into production environments. The strategy combines synthetic data generation algorithms producing statistically representative datasets with anonymization techniques enabling safe use of production-derived data shapes for performance characterization. Data freshness maintenance ensures test data remains representative of current production distributions, preventing test blind spots emerging from stale synthetic profiles diverging from evolving real-world patterns.

10.1 Synthetic Data Generation Strategies

Synthetic data generation creates artificial test data preserving statistical properties of real data without containing actual sensitive information. Generation strategies range from simple random value substitution within valid format constraints through sophisticated machine learning models capturing complex correlations and distribution patterns observed in production data. Domain-specific generators produce realistic security event data including properly formatted IP addresses, domain names, file hashes, and timestamp sequences respecting chronological ordering and inter-field relationships expected by parsing and

correlation logic under test.

Data Type	Generation Method	Tool/Library	Volume	Realism Level
User Records	Faker library templates	Faker (Python/JS)	10K records	High
Security Events	Template + randomization	Custom generator	100K events/day	Very High
Network Logs	PCAP synthesis	tcpreplay, Custom	1 GB/hour	Very High
Authentication Logs	Pattern-based generation	Custom scripts	50K entries	High
Threat Intel	STIX template population	python-stix2	5K IOCs	High
Case Data	Workflow state machines	Factory pattern	2K cases	Medium-High
Performance Data	Statistical distribution fit	NumPy, pandas	Variable	Very High

Table 10.1: Synthetic Data Generation Strategies

11. Defect Tracking and Remediation

Defect tracking manages the complete lifecycle from bug discovery through resolution verification, providing visibility into quality trends, enabling root cause analysis for systemic issues, and documenting evidence of remediation for audit and compliance requirements. The defect management process classifies findings by severity driving response priorities, assigns ownership accountable for resolution delivery, tracks aging to prevent indefinite postponement of difficult fixes, and measures defect escape rates indicating testing effectiveness at catching issues before production deployment.

11.1 Severity Classification and SLAs

Severity	Definition	Response SLA	Resolution SLA	Escalation
Critical (P0)	Production down, data loss, security breach	1 hour	24 hours	Immediate: CTO
Major (P1)	Core feature broken, significant workaround	4 hours	3 days	4 hours: VP Eng
Moderate (P2)	Feature impaired, easy workaround exists	1 business day	2 sprints	3 days: Director
Minor (P3)	Cosmetic issue, edge case, enhancement	1 week	Backlog priority	Sprint planning
Trivial (P4)	Documentation, typos, nice-to-have	Next available	When convenient	None required

Table 11.1: Defect Severity Classification and SLAs

12. Release Readiness Checklist

The release readiness checklist provides a comprehensive go/no-go decision framework ensuring all quality, security, operational, and business criteria are satisfied before promoting software to production environments. Checklist items span functional completeness verification, quality metric threshold validation, security assessment clearance, operational readiness confirmation, and stakeholder sign-off collection. Each item requires explicit acknowledgment with evidence references supporting affirmative responses, creating auditable documentation of release due diligence.

12.1 Complete Release Readiness Checklist

#	Checklist Item	Category	Owner	Status	Evidence
1	All P0/P1 defects resolved or deferred with approval	Quality	QA Lead	[ ]	Jira query
2	Unit test coverage > 85% (new code > 90%)	Quality	Tech Lead	[ ]	Codecov report
3	Integration tests passing (> 98% pass rate)	Quality	DevOps	[ ]	CI pipeline
4	E2E critical paths validated in staging	Quality	QA Lead	[ ]	Playwright
5	Performance benchmarks meet SLA targets	Performance	Perf Eng	[ ]	K6 report
6	No CRITICAL/HIGH security vulnerabilities	Security	AppSec	[ ]	Scan results
7	Penetration test completed (annual/major release)	Security	Sec Ops	[ ]	Pentest report
8	Dependencies scanned, no known exploitable CVEs	Security	AppSec	[ ]	Snyk report
9	Infrastructure Terraform reviewed and applied	Operations	SRE	[ ]	Plan output
10	Database migrations tested and backward compatible	Operations	DBA	[ ]	Migration script
11	Runbooks updated for new features	Operations	SRE	[ ]	Confluence
12	On-call team briefed on release changes	Operations	Team Lead	[ ]	Meeting notes
13	Rollback procedure tested and documented	Operations	SRE	[ ]	Runbook
14	Feature flags ready for gradual rollout	Release	PM	[ ]	Config
15	Customer-facing documentation updated	Documentation	Tech Writer	[ ]	Docs site
16	Legal/compliance review completed (if required)	Compliance	GRC	[ ]	Approval email
17	Executive sign-off obtained for GA release	Business	VP Product	[ ]	Email/thread

Table 12.1: Production Release Readiness Checklist